

Agenda:

- Basics of programming
- Fundamentals of python
- Variables, Data Types
Operators
- Console IO

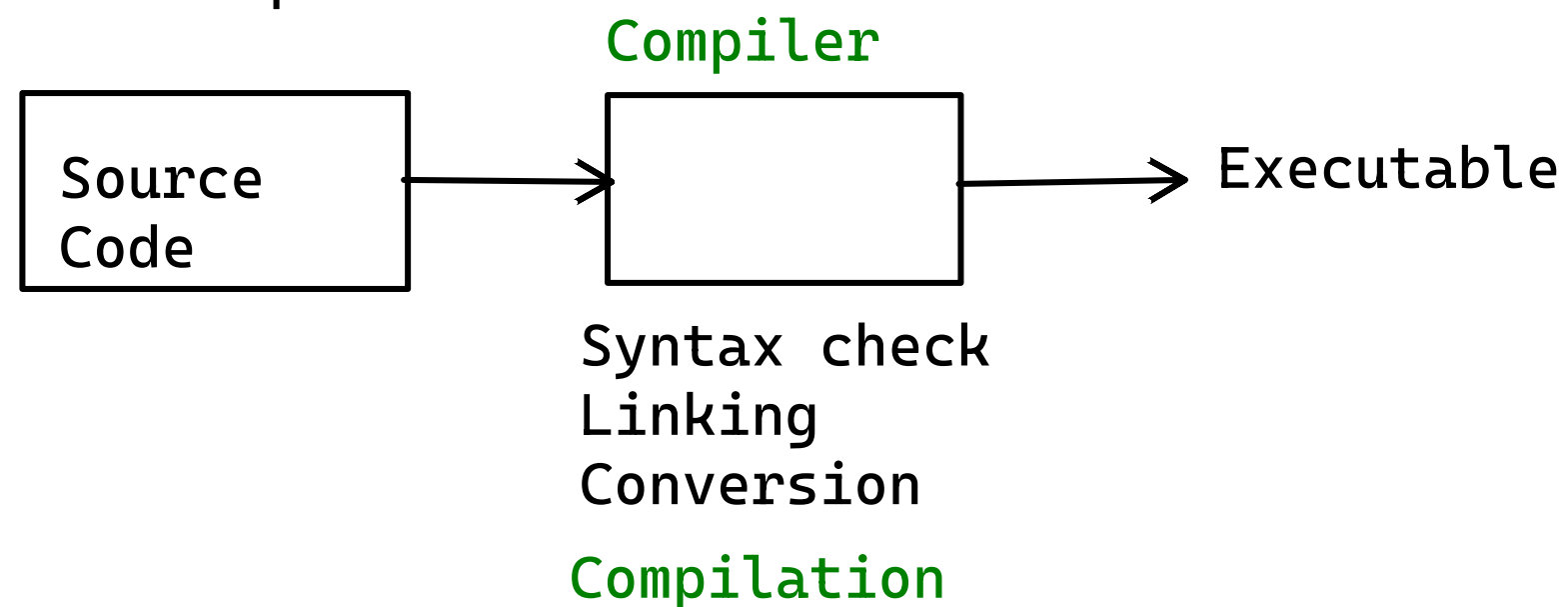
Timings:

9.30 to 11.15AM - P1
11.15 to 11.30 - break
11.30 to 1.15 -p2
1.15 to 2.15 - lunch break
2.15 to 3.45 - p3
3.45 to 4.00 - break
4.00 to 5.30 -p4

- 1/0 - Binary Number System
- High Level Language to Machine Level language
- Two:

1. Compilation
2. Interpretation

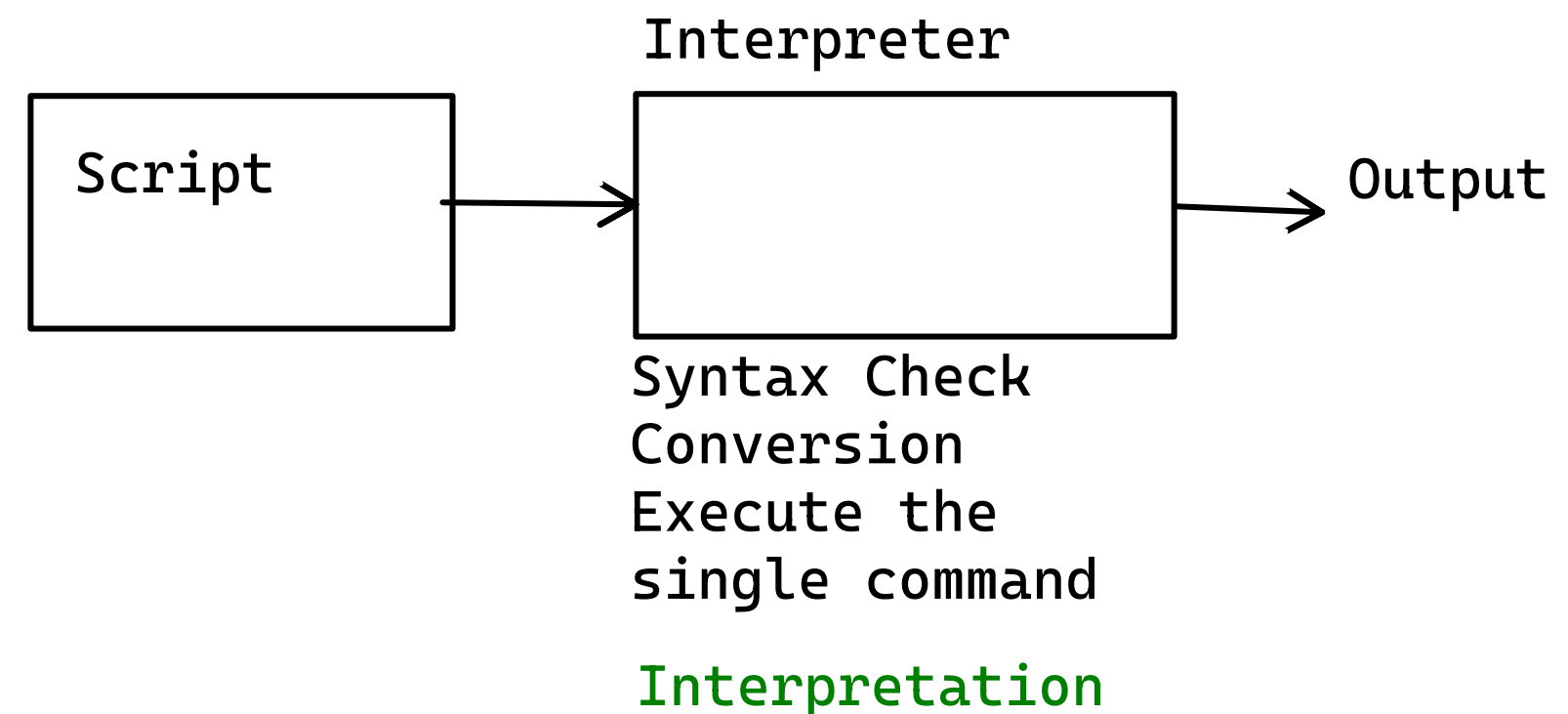
1. Compilation



C, C++, go, rust....

Pros & Cons

- No additional Software
- Secure
- Needs additional time for compilation
- granular control
- Expertise is needed
- Faster



Python, JavaScript....

- Needs Interpreter
- Not secure
- No need of additional time
- less granular control more abstraction
- less faster compare to compiled languages

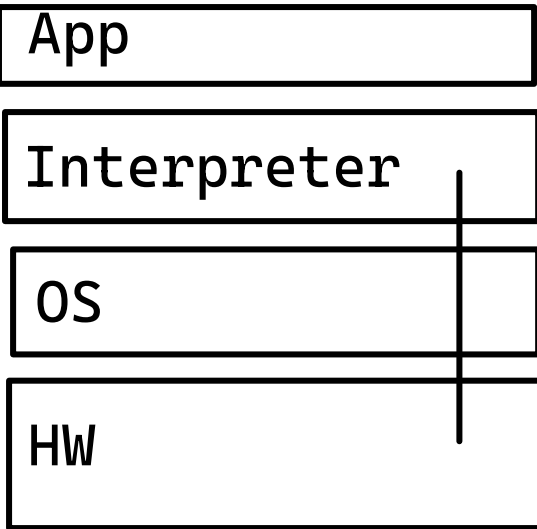
Python:

- Interpreted Language
- Open Source language
- Platform Independent language:App can be run on any combination of HW & OS without modification

Open Source:

Source Code is available – even for modification & Redistribution
GPL,MIT, APACHE LICENSE

WORA



Guido Van Rossum

Interpreter is Dependent on Architecture

WIN/LIN/MAC/ ...

... 128/64/32/16/8.....

- General purpose language: System Apps, Web Applications, CLIs, ML/AI, DS....
- Dynamically Typed Language (later)

Versions of Python

2.x.x
3.x.x (latest)

2.x.x ~~3.x.x~~

3.9.15 3.12.12 3.15.3

Interpreters of Python

Cython - C+Python
Jython - Java + Python
RPython - R + Python
Pypi - python interpreter written in python

Lab: Installation of Python

How to learn a programming language

- How to store temporary data
- How to do conditions
- How to repeat
- What is the base paradigm

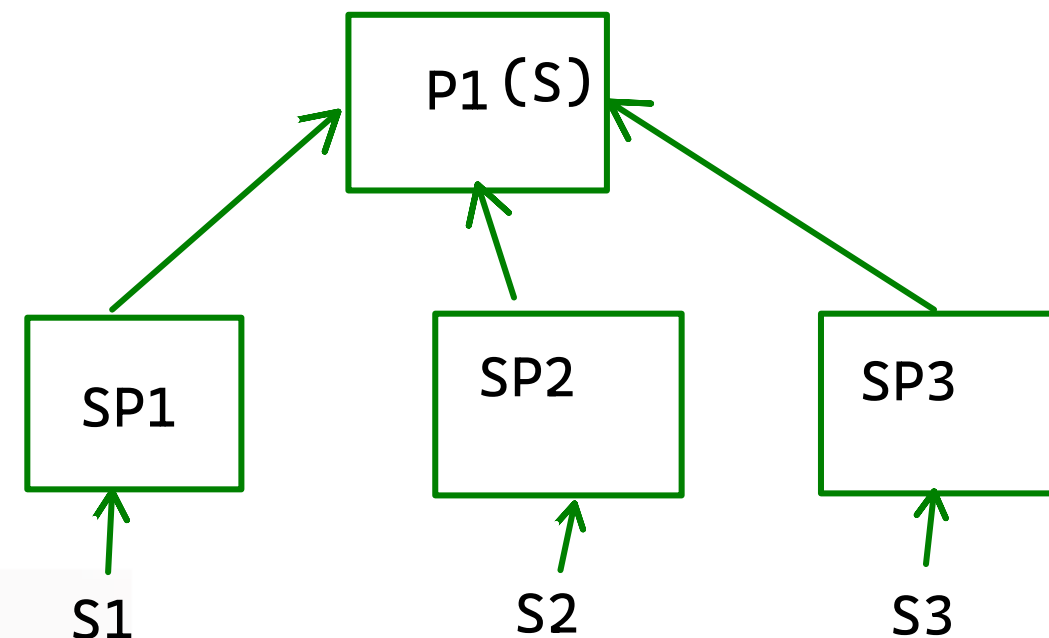
Base Paradigm of python

- Functional programming language

Other Paradigms:

- Object Oriented
- procedural

Functional Programming language



Example:

Scenario: You are a staff, You get CSVs or data from various dept
You have to do analysis, compile, provide a report for the
data received - HTML/PDF

Problem:

You are a staff, You get CSVs or data from various dept
You have to do analysis, compile, provide a report for the
data received – HTML/PDF

Sub Problems:

How to read the data	- read using python pandas library
Clean the data	- Missing data analysis, fill with default values
How to perform analysis	- Computations to be done
How to generate report	- Reportlab/fpdf.... tool to generate report

Ways of interaction with Python

- Interactive Mode
 - Script Mode (.py)
 - Notebook Mode/Jupyter Notebook/ Google Colab (.ipynb)
- Data Analyst / Scientist

How to Store Data Temporarily

Variable: Place holder to data

Variable Identifier	Assignment operator	Actual Value
price	=	23.45

```
price  
print(price)
```

Rules:

- alpha_numeric or _
- Cannot start with a number
- must not be a keyword

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
```

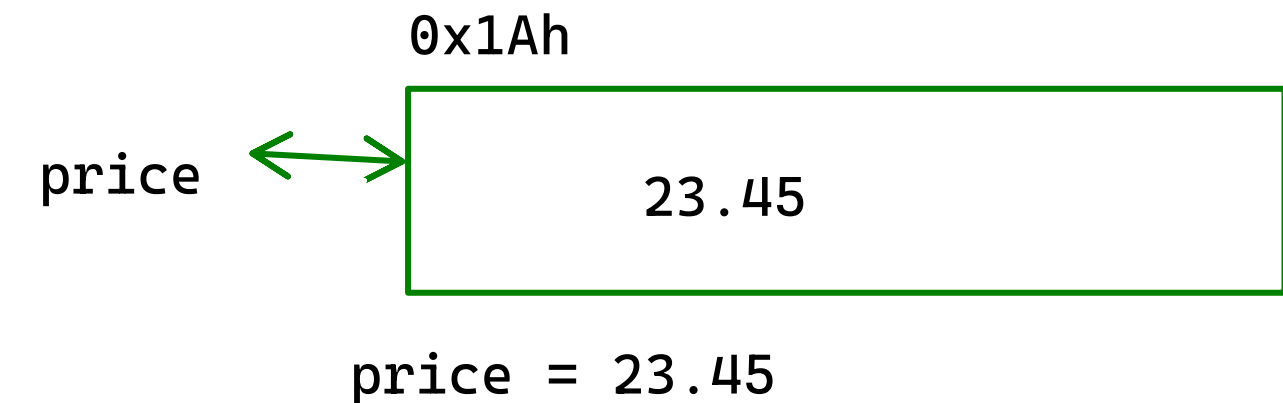
Conventions:

- Have meaningful names
- snake_case

Question: Identify Valid/ Invalid

- | | |
|------------|----------|
| 1. vishwas | 6. _123 |
| 2. var | 7. _abc |
| 3. else1 | 8. abc_! |
| 4. def | |
| 5. !123 | |

1. Valid
2. Valid
3. Valid
4. Invalid
5. Invalid
6. Valid
7. Valid
8. Invalid



```
>>> price = 23.45  
>>> price  
23.45  
>>> print(price)  
23.45  
>>> type(price)  
<class 'float'>  
>>> age = 18  
>>> age  
18  
>>> type(age)  
<class 'int'>  
>>> name = "Vishwas K Singh:  
File "<python-input-7>", line 1  
    name = "Vishwas K Singh:  
          ^  
SyntaxError: unterminated string literal (detected at line 1)  
>>> name = "Vishwas K Singh"  
>>> name  
'Vishwas K Singh'  
>>> type(name)  
<class 'str'>  
>>>
```

Data Types

- int
- float
- str
- bool
- complex

```
>>> v8 = 23.45
>>> type(v8)
<class 'float'>
>>> v9 = 2345e-100
>>> v9
2.345e-97
>>> v9 = 2345e-2
>>> v9
23.45
>>> v10 = 2345e-2
>>> v10
23.45
>>> v11 = .02345e2
>>> v11
2.345
>>> v11 = .2345e2
>>> v11
23.45
>>>
```

```
>>> v1 = "vishwas"
>>> type(v1)
<class 'str'>
>>> v2 = 1234
>>> type(v2)
<class 'int'>
>>> v3 = 1234.5
>>> type(v3)
<class 'float'>
>>> v4 = True
>>> type(v4)
<class 'bool'>
>>> v5 = 23+6j
>>> type(v5)
<class 'complex'>
>>> |
```

```
<class 'complex'>
>>> v6 = 12345678908765432134567890987654321234567890987654321234567898765432457898765432
>>> v6
12345678908765432134567890987654321234567890987654321234567898765432457898765432
>>> v7 = 0b1010
>>> v7
10
>>> type(v7)
<class 'int'>
>>> v7 = 0x1a
>>> v7
26
>>> type(v7)
<class 'int'>
>>> v7 = 0o1234
>>> v7
668
>>> type(v7)
<class 'int'>
>>> v7 = 30_00_000
>>> v7
3000000
>>> type(v7)
<class 'int'>
```

Strings

```
>>> type(s1)
<class 'str'>
>>> s2 = 'vishwas'
>>> s2
'vishwas'
>>> type(s2)
<class 'str'>
>>> s3 = """Vishwas"""
>>> s3
'Vishwas'
>>> type(s3)
<class 'str'>
>>> s4 = '''Vishwas'''
>>> s4
'Vishwas'
>>> type(s4)
<class 'str'>
>>> s5 = "vishwas's book"
>>> s5
"vishwas's book"
>>> s6 = 'vishwas said "I am Vishwas, I am learning Python"'
>>> s6
'vishwas said "I am Vishwas, I am learning Python"'
>>> s7 = '''This is an example fo multiline string.
... This was created using triple quotes.
... This is a common scenario when text processing'''
>>> s7
'This is an example fo multiline string.\nThis was created using triple quotes.\nThis is a common scenario when text pro
cessing'
>>> |
```


Special Strings

```
>>> s10 = "C:\Users\VishwasKSingh\Workspace\ey-coh6-workspace\README.md"
File "<python-input-86>", line 1
    s10 = "C:\Users\VishwasKSingh\Workspace\ey-coh6-workspace\README.md"
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
SyntaxError: (unicode error) 'unicodeescape' codec can't decode bytes in position 2-3: truncated \UXXXXXXX escape
>>> s10 = r"C:\Users\VishwasKSingh\Workspace\ey-coh6-workspace\README.md"
>>> s10
'C:\\Users\\VishwasKSingh\\Workspace\\ey-coh6-workspace\\README.md'
>>> name = "viswhas"
>>> s11 = f"This is {name}'s book"
>>> s11
"This is viswhas's book"
>>> |
```

Operators

```
>>> 2+2
4
>>> 2-1
1
>>> 4*3
12
>>> 7/2
3.5
>>> 7//2
3
>>> 7%2
1
>>> 2**3
8
>>> |
```

```
>>> 2 > 3
False
>>> 3 > 4
False
>>> 3 < 5
True
>>> 3 >= 2
True
>>> 3 <= 5
True
>>> 3 != 5
True
>>> 4 == 4
True
>>> |
```

```
>>> 2 > 3 and 3 < 5
False
>>> 8 != 8 and 5 < 6
False
>>> |
```

- Non Zero value is always true
- 0 is always false

2 and 3	2 or 4	0 or 9
2 and 5	4 or 2	9 or 0
5 and 2	9 or 2	0 and 9
5 and 3	2 or 9	9 and 0

-1 and 5
5 and -1

Short circuit evaluation

– And Condition

expr1 and expr2

expr2 is evaluated only when expr1 is true

– Or Condition

expr1 or expr2

expr2 is not evaluated at all if expr1 is true

PEDMAS

 $23 / 6 * 4 + 3 ** (2 * 2)$

Console I/O

– I can take a value from the user using console

– input(prompt) → is a function to do that

– it always returns a str

– you have to convert it explicitly

```
>>> print("Vishwas","k","singh")
Vishwas k singh
>>> print("Vishwas","k","singh", sep="*")
Vishwas*k*singh
>>> print("Vishwas","k","singh", sep="*", end="---")
>>> was*k*singh---
```

```
>>> x = 10
>>> y = 2
>>> x / y
5.0
>>> y = 0
>>> x / y
Traceback (most recent call last):
  File "<python-input-124>", line 1, in <module>
    x/y
    ~^~
ZeroDivisionError: division by zero
>>> y != 0 and x / y
False
>>> y = 2
>>> y != 0 and x / y
5.0
>>> |
```

1. Simple Interest Formula: $si = (p*t*r)/100$

2. SIP vs Lumpsum calculator

3.

Users	City
-------	------

Vishwas	Pune
---------	------

Raju	Goa
------	-----

Sachin	Delhi
--------	-------

4. Brokerage Calculator

5. Income Tax Calculator

Day 1 Coverage

- Basics of programming
- Fundamentals of python
- Variables, Data Types
Operators
- Console IO

What to expect

- Conditionals
- Loops
- Str
- List, Dict, Tuple Set

Day2 Agenda

- Conditionals
- Loops
- DS

Conditionals

if
if-else
if-elif-else
nested-if

Block of Code

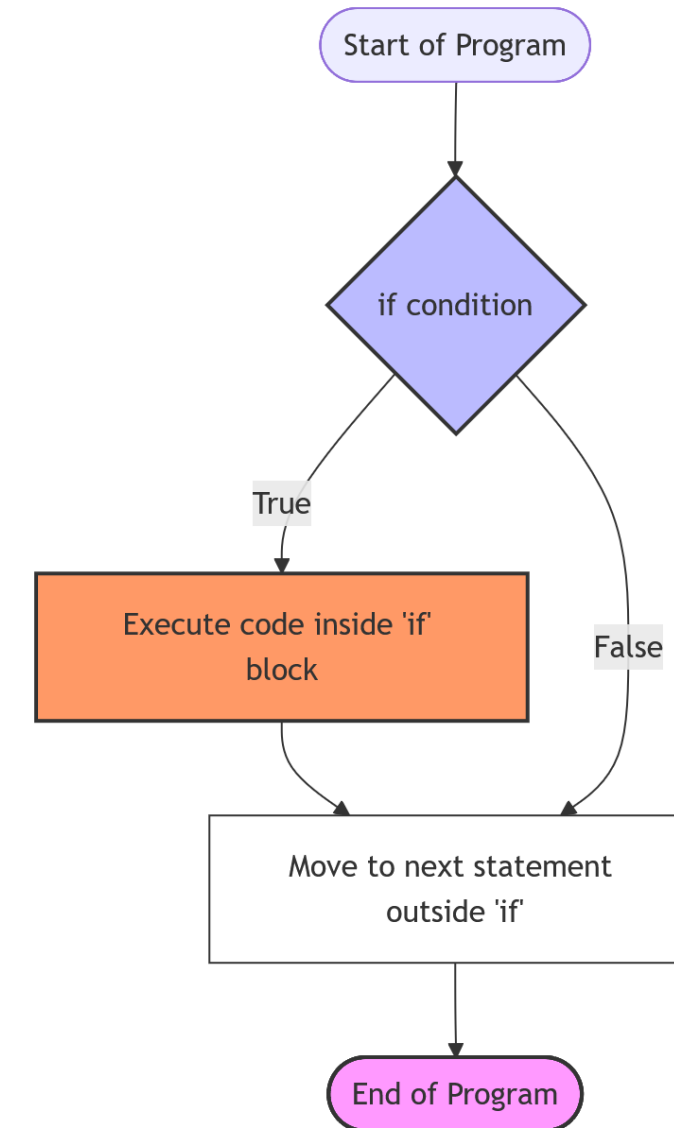
```
_____  
:      s1  
      s2  
      s3  
sn1  
sn2
```

```
>>> x = 10  
>>> if x > 10:  
...     print("x is greater than 10")  
...  
>>> if x < 10:  
...     print("x is less than 10")  
...  
>>> if x <= 10:  
...     print("x is less than or equal to 10")  
...  
x is less than or equal to 10  
>>>
```

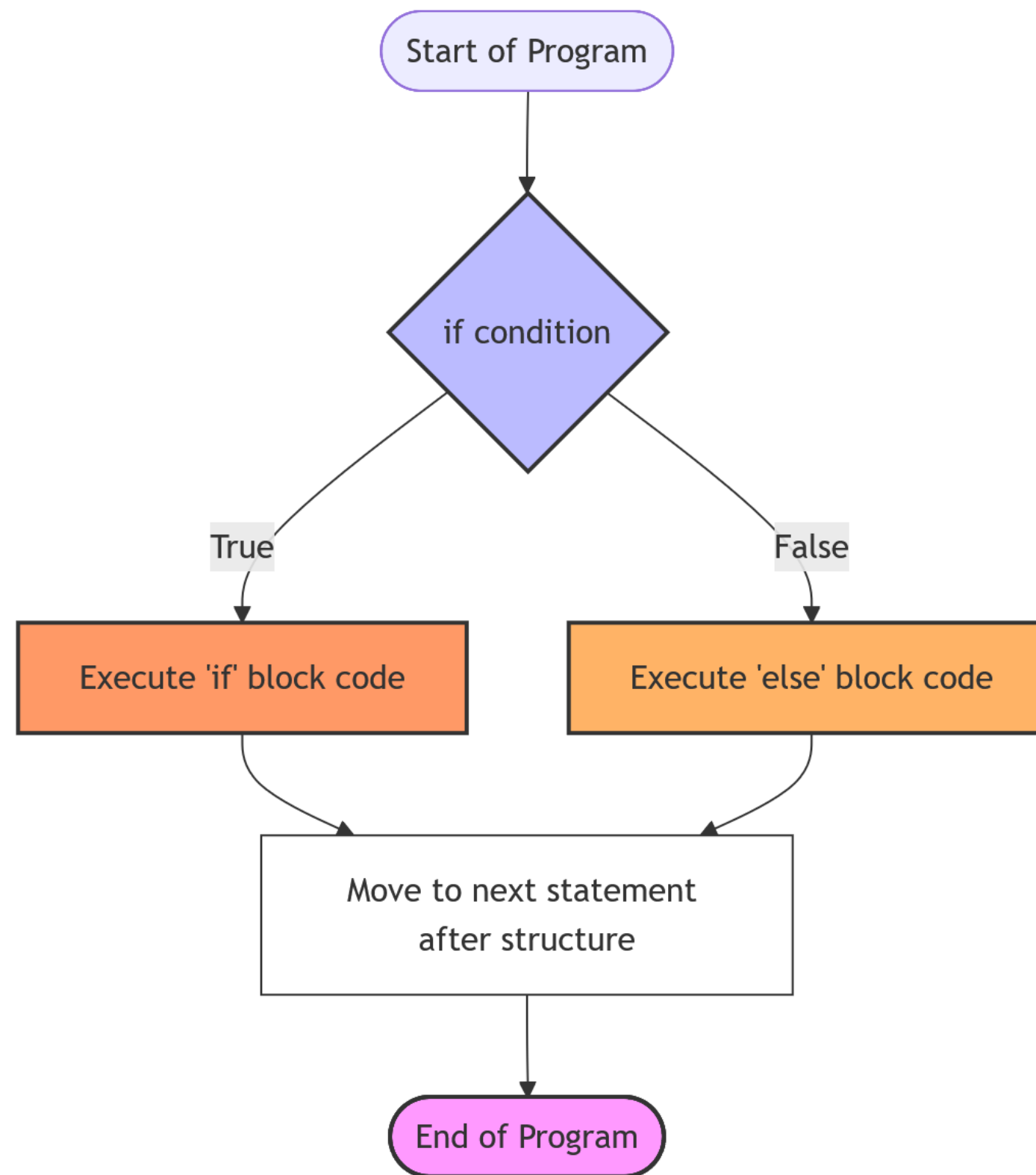
if condn:

s1
s2
s3

sn1
sn2



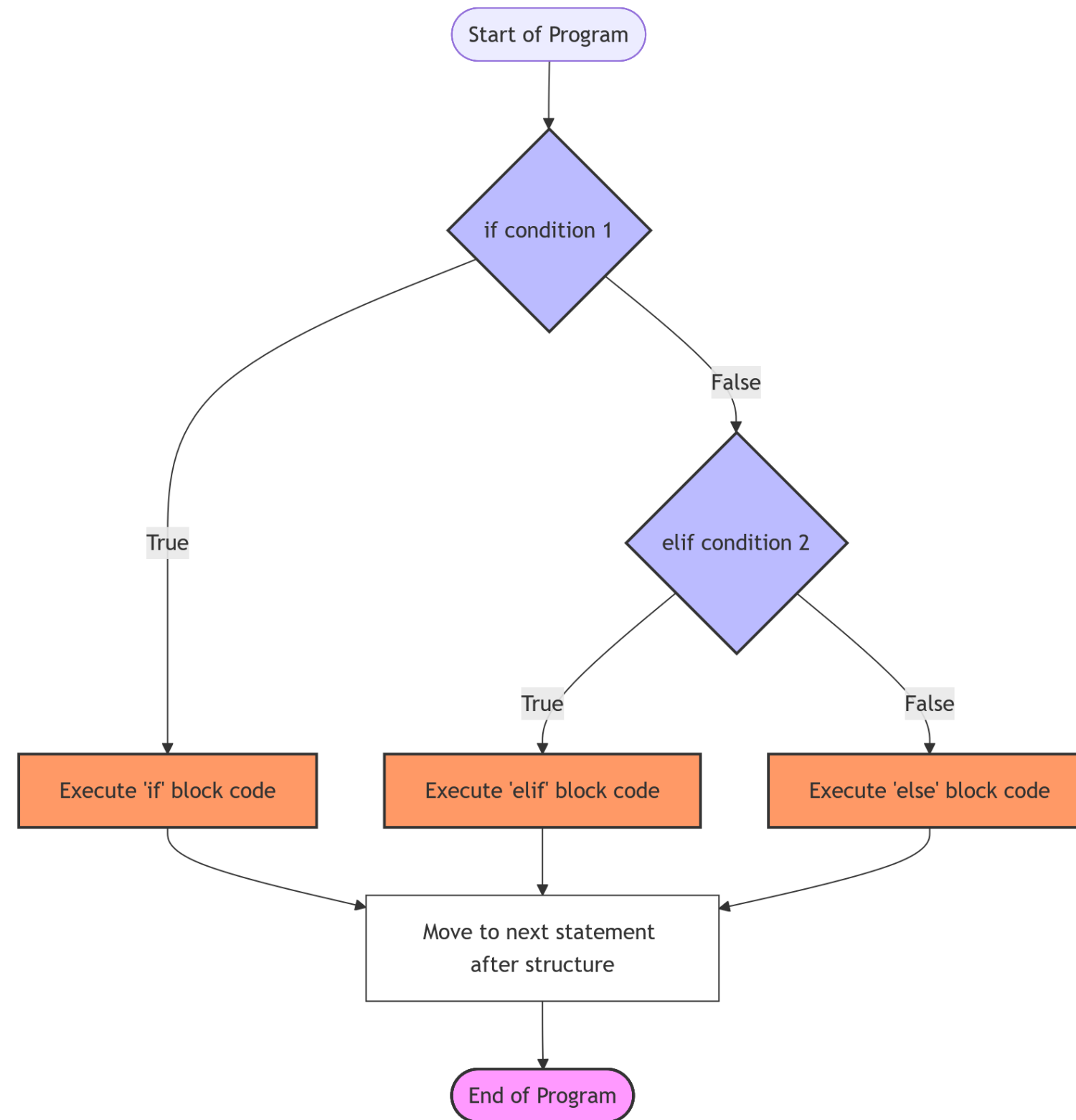
```
if condn:  
    s1  
    s2  
else:  
    s3  
    s4  
sn1  
sn2
```



```
>>> x = 10  
>>> if x == 10:  
...     print("x is equal to 10")  
... else:  
...     print("x is not equal to 10")  
...  
x is equal to 10  
>>>
```

if -elif-else

```
if condn1:  
    s1  
    s2  
elif condn2:  
    s3  
    s4  
else:  
    s5  
    s6  
sn1  
sn2
```



```
>>> if x >=56:  
...     print("x is greater than or equal to 56")  
... elif x <= 46:  
...     print("x is less than or equal to 46")  
... else:  
...     print("x is between 46 & 56")  
...  
x is greater than or equal to 56  
>>>
```


What will be printed

```
x = 50
if x > 40:
    print("x>40")
elif x > 45:
    print("x>45")
elif x > 47:
    print("x > 47")
else:
    print("x < 100")
```

A. `print("x>40")`
`print("x>45")`
`print("x > 47")`
`print("x < 100")`

B. `print("x>45")`
`print("x > 47")`
`print("x < 100")`

C. `print("x < 100")`

D. `print("x>40")`

E. `print("x > 47")`

Lab: Calculate Income tax
based on the tax slab

ANNUAL INCOME	WILL BE TAXED AT...
Up to ₹2.5 lakh	NII
₹2.5 lakh to ₹5 lakh	5%
₹5 lakh to ₹7.5 lakh	10%
₹7.5 lakh to ₹10 lakh	15%
₹10 lakh to ₹12.5 lakh	20%
₹12.5 lakh to ₹15 lakh	25%
Above ₹15 lakh	30%

Loops

Repeat block of statements

For Loops

```
for i in range(n):  
    print(i)
```

`range(stop)` -> range object

`range(start, stop[, step])` -> range object

Return an object that produces a sequence of integers from start (inclusive) to stop (exclusive) by step. `range(i, j)` produces `i, i+1, i+2, ..., j-1`. start defaults to 0, and stop is omitted! `range(4)` produces 0, 1, 2, 3. These are exactly the valid indices for a list of 4 elements. When step is given, it specifies the increment (or decrement).

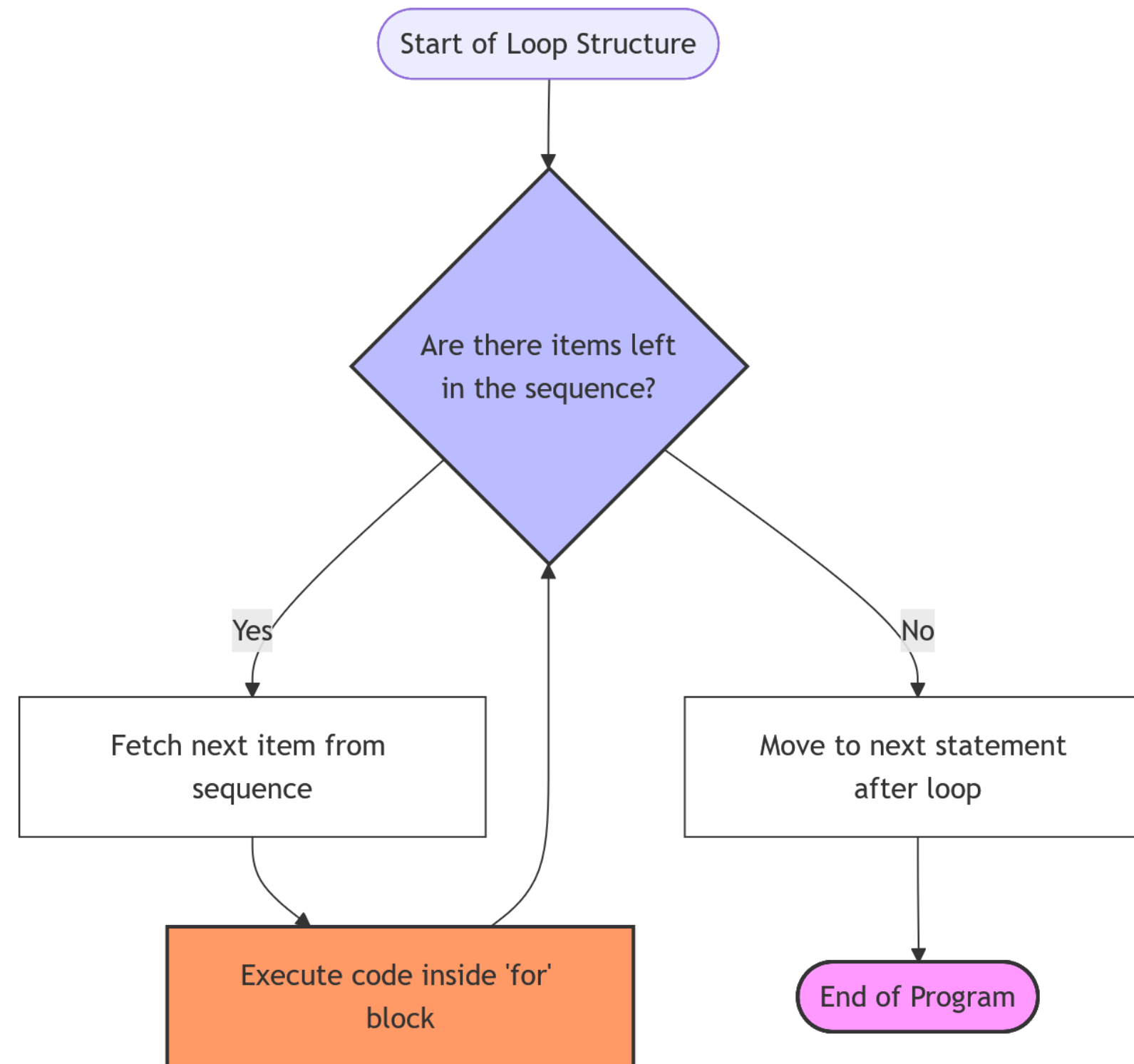
```
>>> for i in range(11):  
...     print(i)  
...  
0  
1  
2  
3  
4  
5  
6  
7  
8  
9  
10  
>>>  
>>> for i in range(1,11):  
...     print(i)  
...  
1  
2  
3  
4  
5  
6  
7  
8  
9  
10  
>>> |
```

While Loops

```
x = 10
```

```
while x > 0:  
    print(x)  
    x -= 1
```

```
>>> x = 10  
>>> while x > 0:  
...     print(x)  
...     x -= 1  
10  
9  
8  
7  
6  
5  
4  
3  
2  
1  
>>>
```



Strings

- Immutable (cant be modified)
 - Text Processing
 - Indexes
 - Sequence of Characters
 - Slicing
- Always from left to right
EXCEPT WHEN STEP IS -VE

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	
	V	I	S	H	W	A	S		K		S	I	N	G	H	
	-15	-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1	

```
>>> S1 = "VISHWAS K SINGH"
>>> len(S1)
15
>>> S1[1]
'I'
>>> S1[2]
'S'
>>> S1[14]
'H'
>>> S1[-1]
'H'
>>> S1[-2]
'G'
>>> S1[-11]
'W'
>>> S1[-15]
'V'
```

```
>>> S1[0:7]
'VISHWAS'
>>> S1[0:15]
'VISHWAS K SINGH'
>>> S1[10:15]
'SINGH'
>>> S1[10:14]
'SING'
>>> S1[10:200]
'SINGH'
>>> S1[-15:-8]
'VISHWAS'
>>> S1[-5:-2]
'SIN'
>>> S1[-5:0]
''
>>> S1[-5:-1]
'SING'
>>> S1[-5:]
'SINGH'
>>> S1[: -8]
'VISHWAS'
```

```
>>> S1[0:-8]
'VISHWAS'
>>> S1[-15:7]
'VISHWAS'
>>> S1[10:]
'SINGH'
>>> S1[-5:]
'SINGH'
>>> S1[10:-2]
'SIN'
>>> S1[-5:13]
'SIN'
```

```
>>> S1[:]
'VISHWAS K SINGH'
>>> S1[::2]
'VSWSKSNH'
>>> S1[::3]
'VHS N'
```

```
>>> S1[::-1]
'HGNIS K SAWHSIV'
>>> S1[::-2]
'HNSKSWSV'
>>> S1[::-3]
'HIKAS'
>>>
```

```
>>> S1[6:3:-1]
'SAW'
>>> S1[-9:-12:-1]
'SAW'
>>> S1[6:-12:-1]
'SAW'
>>> S1[-9:3:-1]
'SAW'
```

String Methods

```
>>> dir(str)
['_add__', '__class__', '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getat
tribute__', '__getitem__', '__getnewargs__', '__getstate__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__i
ter__', '__le__', '__len__', '__lt__', '__mod__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr
__', '__rmod__', '__rmul__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', 'capitalize', 'casefold', 'cent
er', 'count', 'encode', 'endswith', 'expandtabs', 'find', 'format', 'format_map', 'index', 'isalnum', 'isalpha', 'isasci
i', 'isdecimal', 'isdigit', 'isidentifier', 'islower', 'isnumeric', 'isprintable', 'isspace', 'istitle', 'isupper', 'joi
n', 'ljust', 'lower', 'lstrip', 'maketrans', 'partition', 'removeprefix', 'removesuffix', 'replace', 'rfind', 'rindex',
'rjust', 'rpartition', 'rsplit', 'rstrip', 'split', 'splitlines', 'startswith', 'strip', 'swapcase', 'title', 'translate
', 'upper', 'zfill']
```

```
>>> S1
'VISHWAS K SINGH'
>>> S1.lower()
'vishwas k singh'
>>> S1.capitalize()
'Vishwas k singh'
```

```
>>> S1.center(80, "-")
'-----VISHWAS K SINGH-----'
>>> S1.rjust(80, "-")
'-----VISHWAS K SINGH'
>>> S1.ljust(80, "-")
'VISHWAS K SINGH-----'
>>> S1.swapcase()
'vishwas k singh'
>>> S1.replace("SINGH", "*****")
'VISHWAS K *****'
>>> S1.startswith("V")
True
```


Lab:

1. Username extractor:

email: vishwas@cloudthat.com

username: vishwas

2. valid email:

email: vishwas@cloudthat.com

Vaild

email: vishwas@cloudthat@.com

Invalid

email: vishwas

Invalid

3. Valid IPv4 Address

ip: 192.168.1.1

Valid

ip: a.b.192.1

Invalid

ip: 279.172.2.1

Invalid

4. private / public ipv4

Ipv4:

- 4 Fields
- Dotted Decimal Notation
- 0 to 255

RFC 1918 name	IP address range
24-bit block	10.0.0.0 – 10.255.255.255
20-bit block	172.16.0.0 – 172.31.255.255
16-bit block	192.168.0.0 – 192.168.255.255

List

- Stores multiple values, duplicates also
- it is mutable
- it is heterogeneous
- indexing, slicing

```
>>> l1 = []
>>> l2 = list()
>>> l3 = ["apple", "Mango", 1, 2, 34, {1: "Vishwas"}, ("Pune", "Goa")]
>>> l3[0]
'apple'
>>> l3[1]
'Mango'
>>> dir(list)
['__add__', '__class__', '__class_getitem__', '__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattr__', '__getitem__', '__getstate__', '__gt__', '__hash__', '__iadd__', '__imul__', '__init__', '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__reversed__', '__rmul__', '__setattr__', '__setitem__', '__sizeof__', '__str__', '__subclasshook__', 'append', 'clear', 'copy', 'count', 'extend', 'index', 'insert', 'pop', 'remove', 'reverse', 'sort']
>>>
```


Dict

- Key Value
- Key must be unique
- Heterogeneous
- Key can be any data type, int & str recommended

```
>>> d1 = {}
>>> type(d1)
<class 'dict'>
>>> d2 = dict()
>>> type(d2)
<class 'dict'>
>>> d3 = {"user": "vishwas", "city": "pune", "company": "ct", 1: "Apple", 2: "mango"}
>>> d3
{'user': 'vishwas', 'city': 'pune', 'company': 'ct', 1: 'Apple', 2: 'mango'}
>>> d3["user"]
'vishwas'
>>> d3["city"]
'pune'
>>> d3[5]="banana"
```

```
>>> dir(dict)
['__class__', '__class_getitem__', '__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__', '__eq__', '__form
at__', '__ge__', '__getattribute__', '__getitem__', '__getstate__', '__gt__', '__hash__', '__init__', '__init_subclass__
', '__ior__', '__iter__', '__le__', '__len__', '__lt__', '__ne__', '__new__', '__or__', '__reduce__', '__reduce_ex__', '__
repr__', '__reversed__', '__ror__', '__setattr__', '__setitem__', '__sizeof__', '__str__', '__subclasshook__', 'clear'
, 'copy', 'fromkeys', 'get', 'items', 'keys', 'pop', 'popitem', 'setdefault', 'update', 'values']
>>>
```


Lab: Given a list of ip_address remove invalid ip addresses without filter function
["123.1.25.1", "72.24.1.6", "a.b", "192.168", "192.168.1.1"]

Lab: Frequency Counter

"vishwask Singh"

"v": 1, "i": 2, "s": 3, "h": 2.....

Tuple

- Immutable
- acts like a list - indexed, sliced, heterogeneous
- Python uses this in backend for passing the data

```
>>> t1 = ("Apple", "Mango", "grapes", 1, 2, 3, "grapes", {1: "vishwas", 2: "arjun"})
>>> t1[0]
'Apple'
>>> t1[-1]
{1: 'vishwas', 2: 'arjun'}
>>> t1[:4]
('Apple', 'Mango', 'grapes', 1)
>>>
```

```
>>> t2 = "pune", "mumbai", "goa", "delhi"
>>> t2
('pune', 'mumbai', 'goa', 'delhi')
>>> a, b, c, d = t2
>>> a
'pune'
>>> b
'mumbai'
>>> c
'goa'
>>> d
'delhi'
>>>
```

```
>>> t3 = (1, 2, 3, 4, 5)
>>> t4 = (1, 2, 3, 3, 7)
>>> t3 > t4
True
>>> t3 < t4
False
>>> t3.index(4)
3
>>> t4.count(3)
2
>>>
```

Set

- Distinct, homogeneous

```
>>> s1 = {45,67,89,1,2,3,4,45,6,7,89}
>>> s1
{1, 2, 3, 67, 4, 6, 7, 45, 89}
>>> type(s1)
<class 'set'>
>>> s2 = s1.copy()
>>> s2
{1, 2, 3, 67, 4, 6, 7, 45, 89}
>>> s1.clear()
>>> s1
set()
>>> s1.add("vishwas"
... )
>>> s1
{'vishwas'}
>>> s1.add("arjun")
>>> s1
{'arjun', 'vishwas'}
>>> dir(set)
['_and__', '__class__', '__class_getitem__', '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__',
'__ge__', '__getattr__', '__getstate__', '__gt__', '__hash__', '__iand__', '__init__', '__init_subclass__', '__ior__',
'__isub__', '__iter__', '__ixor__', '__le__', '__len__', '__lt__', '__ne__', '__new__', '__or__', '__rand__', '__r
reduce__', '__reduce_ex__', '__repr__', '__ror__', '__rsub__', '__rxor__', '__setattr__', '__sizeof__', '__str__', '__su
b__', '__subclasshook__', '__xor__', 'add', 'clear', 'copy', 'difference', 'difference_update', 'discard', 'intersection
', 'intersection_update', 'isdisjoint', 'issubset', 'issuperset', 'pop', 'remove', 'symmetric_difference', 'symmetric_di
fference_update', 'union', 'update']
>>>
```

Lab:

1. Given a list which contains duplicates, create a list with distinct items
2. Given two strings
s1 = "abbadda", s2= "abcbedf"
output 2 strings where op1 will contain characters in s1 which are not in s2
op2 will contain characters in s2 which are not in s1

Lab: Given a log data as a list of strings. Which contains data about login attempts by a email id. based on the number of login attempts, find out the email ids which may be marked as suspicious.

Assumption: Number of login attempts > 2 (mark it as suspicious)

Day 2 Summary

- Conditionals
- Loops
- DS

What to expect tomorrow:

- Functions
- Modules
- Intermediate concepts: comprehensions, iterators, generators
- oops
- file handling & exceptions

Day 3 Agenda

- Functions
- Modules
- Intermediate concepts: comprehensions, lambda expr, iterators, generators
- oops
- file handling & exceptions

Functions

- first class citizens
- pass functions as variables

```
def func():  
    statements
```

- may or may not take parameters
- - " '- return values
- can also have variable parameters & multiple return values

Scope of variables

- Local - block
- Global - throughout the script
- priority is given to local variables

Parameter Types

- positional
- keywords or named

Variable arguments

- *args, **kwargs

Lab: Convert / Create a utils.py where you define functions for

1. username_extractor
2. email_validator
3. ip_address_validator
4. private_address_validator

modules

- A file is a module in python

Note: The global statements are also executed when we are importing a module. To address this behaviour of python we include the statements which has to be run only when the file is called directly inside a `if __name__ == '__main__'` block

Package

- Is a collection of modules
- to make a folder as a package, include `__init__.py`

Doc Strings & Type hinting

List comprehension

generate a list of 1000 integers

```
>>> l1 = []
>>> for i in range(1000):
...     l1.append(i)
...
>>> l1
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13,
2, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43,
2, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73,
2, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102,
118, 119, 120, 121, 122, 123, 124, 125, 126,
```

Normal Approach

```
>>> l2 = [x for x in range(1000)]
>>> l2
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14,
2, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44,
2, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74,
2, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103,
118, 119, 120, 121, 122, 123, 124, 125, 126, 127,
142, 143, 144, 145, 146, 147, 148, 149, 150, 151,
166, 167, 168, 169, 170, 171, 172, 173, 174, 175,
```

```
>>> l3 = [x for x in range(1000) if x%2==0]
>>> l3
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26,
62, 64, 66, 68, 70, 72, 74, 76, 78, 80, 82, 84, 86,
118, 120, 122, 124, 126, 128, 130, 132, 134, 136,
166, 168, 170, 172, 174, 176, 178, 180, 182, 184,
214, 216, 218, 220, 222, 224, 226, 228, 230, 232,
```

List Comprehensions

Lambda Expressions

- Anonymous Functions
- one line function, quick computations

```
lambda x: x**3
```

```
lambda x: x[1]
```

Lab: There is a dictionary of inventory (product & Quantity). Print Sorted items from maximum to minimum quantity

or

You have a dictionary of user emails and number of login attempts. print the users based on login attempts with highest login attempts first

```
data = {  
    "vishwas@cloudthat.com": 1  
    "alice@cloudthat.com":10  
    "kiran@cloudthat.com":3  
    "Riya@cloudthat.com": 15  
}
```

Lab: You have a dictionary of user emails and number of login attempts. extract email ids with exactly 3 attempts

Iterators

- memory efficient

```
>>> l1 = ["apple", "mango", "banana", "grapes"]
>>> l1
['apple', 'mango', 'banana', 'grapes']
>>> it = iter(l1)
>>> next(it)
'apple'
>>> next(it)
'mango'
>>> next(it)
'banana'
>>> next(it)
'grapes'
>>> next(it)
Traceback (most recent call last):
  File "<python-input-7>", line 1, in <module>
    next(it)
    ~~~~^
StopIteration
```

Generator

- Helps to create iterators

```
>>> def gen_num(n):
...     for i in range(n):
...         yield i
...
>>> l = gen_num(100)
>>> print(next(l))
0
>>> print(next(l))
1
>>> print(next(l))
2
>>> print(next(l))
3
>>> print(next(l))
4
>>> print(next(l))
5
```

```
>>> m = (i for i in range(10))
>>> next(m)
0
>>> next(m)
1
>>> next(m)
2
>>> next(m)
3
>>> next(m)
4
>>> next(m)
5
>>> next(m)
6
```

generator expression

Object Oriented Programming

What are the 4 pillars of OOPs?

Encapsulation

```
class Person:
    def __init__(self, name, city):
        self.name = name
        self.city = city

    def __repr__(self):
        return f"Person Name: {self.name}, City: {self.city}"
```

Inheritance

```
class User(Person):
    def __init__(self, name, city, age):
        super().__init__(name, city)
        self.age = age

p1 = Person("vishwas", "Pune")
print(p1)
p2 = Person("Raj", "Delhi")
print(p2)

p3 = User("Riya", "Delhi", 20)
print(p3)
```

Polymorphism

```
class User(Person):
    def __init__(self, name, city, age):
        super().__init__(name, city)
        self.age = age

    def __repr__(self):
        return f"Person Name: {self.name}, City: {self.city}, Age: {self.age}"
```

Abstraction: implemented through abc

- No Access modifiers (Only by convention)

Classes and objects are used to store temporary data & validate it

dataclasses

Decorators

- wrapper to other functions
- which can be called without messing up the code

Examples:

- Common Logging
- Pre authorization
- Time a function

Similar to annotations in Java & attributes in C#

@<FunctionName> or @<ClassName>

```
def my_decorator(func):
    def wrapper(*args,**kwargs):
        # Other Statements
        result = func(*args,**kwargs)
        # Other Statements

        return result
    return wrapper
```

```
1  from datetime import datetime as dt
2
3  # Decorator to see execution time of a function
4  def my_timing(func):
5      def wrapper(*args,**kwargs):
6          start = dt.now()
7          result = func(*args,**kwargs)
8          # Other Statements
9          end = dt.now()
10         print(f"Execution Time: {end-start} ms")
11
12         return result
13     return wrapper
14
15 @my_timing
16 def add(a,b):
17     return a+b
18
19 if __name__ == '__main__':
20     print(add(45,67))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
C:\Users\VishwasKSingh\Workspace\ey-coh6-workspace\utilities>python utils.py
Execution Time: 0:00:00.000024 ms
112
```


Data Classes

- help in reducing boilerplate code
- it provides type hints but not strict enforcements

```
18 from dataclasses import dataclass
19
20 @dataclass
21 class Book:
22     title: str = "UNKNOWN"
23     author: str = "UNKNOWN"
24     pages: int = 0
25
26
27 book1 = Book("Pride and Prejudice", "Jane Austen", 200)
28 print(book1)
29 book2 = Book("Pride and Prejudice", "Jane Austen", "200")
30 print(book1 == book2)
31
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Book(title='Pride and Prejudice', author='Jane Austen', pages=200)
True
```

```
C:\Users\VishwasKSingh\Workspace\ey-coh6-workspace>python 09-dataclasses_usage.py
Book(title='Pride and Prejudice', author='Jane Austen', pages=200)
False
```

File & Exception Handling

Exception Handling

```
=====
Character Meaning
-----
'r'      open for reading (default)
'w'      open for writing, truncating the file first
'x'      create a new file and open it for writing
'a'      open for writing, appending to the end of the file if it exists
'b'      binary mode
't'      text mode (default)
'+'      open a disk file for updating (reading and writing)
=====
```

The default mode is 'rt' (open for reading text). For binary random access, the mode 'w+b' opens and truncates the file to 0 bytes, while 'r+b' opens the file without truncation. The 'x' mode implies 'w' and raises an 'FileExistsError' if the file already exists.

Lab: Read the sample1.txt file using methods read(), readline(), readlines()
observe the difference

Lab: Given a log data in susp_log file, which contains data about login attempts by a email id.
based on the number of login attempts, find out the email ids & IPs which may be marked as suspicious.

Assumption: Number of login attempts > 2 (mark it as suspicious)

Day 3 Summary

- Functions
- Modules & packages
- Iterators, Generator, Lambda Expr, Decorators
- OOPS, Data Classes
- File & Exception handling

What to Expect:

- Built-in Modules re, os, csv, sys, pathlib
- Third-party - pydantic, sqlalchemy, sqlite, pandas, numpy, matplotlib, reportlab/fpdf, jinja2

Built-in Modules

String - Constants for comparing strings
time, date, datetime - Dates & Times
os - Operating System Functions
re - regular expressions
pathlib - OOPs based manipulation of path
os.path - string based manipulation of paths
json - json handling
toml, yaml, csv
venv - virtual env

Lab: Write a word count like program to count number of words, characters, lines, punctuations, digits, vowels in a given file and write it to a summary.log file

```
input - read the file path
count number of characters - read() - len()
count number of lines - readlines() - len()
count number of words - readlines() - for each line split() - len() - add()
count number of punctuations - read() - for each char - string.punctuation
count number of digits - read() - for each char - string.digits
Define vowels - do same as above
```

Lab: Validate the Email addresses

1. It start with a lower case letter
2. It can contain numbers & dot before '@'
3. after '@' it should only contain letters
4. only 1 dot after the 2 chars after '@' after that it can contain only characters

Lab: In the mbox.txt file extract the email ids with lines From:

example: From: stephen.marquard@uct.ac.za



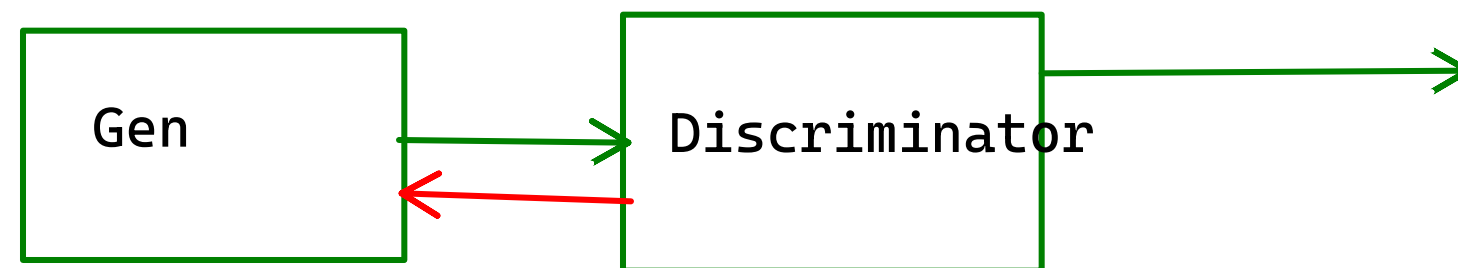
```
app_root/
  config/
    env.json
  src/
    app.py
```

Lab: You are building an application in python, where the configuration information is stored as json in the config directory. You are creating app.py inside src folder. You have to read the configuration information and display results using json. Make sure you do not hard code any paths.

AI assisted Coding

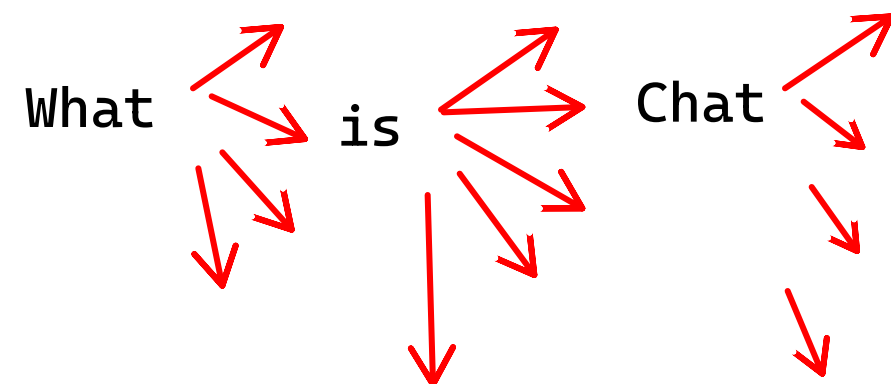
Gen AI inside your browser - ChatGPT, Claude Code, Copilot, Gemini....

GPT - Transformer model



What ~~is~~ ChatGPT

Token → word / Char



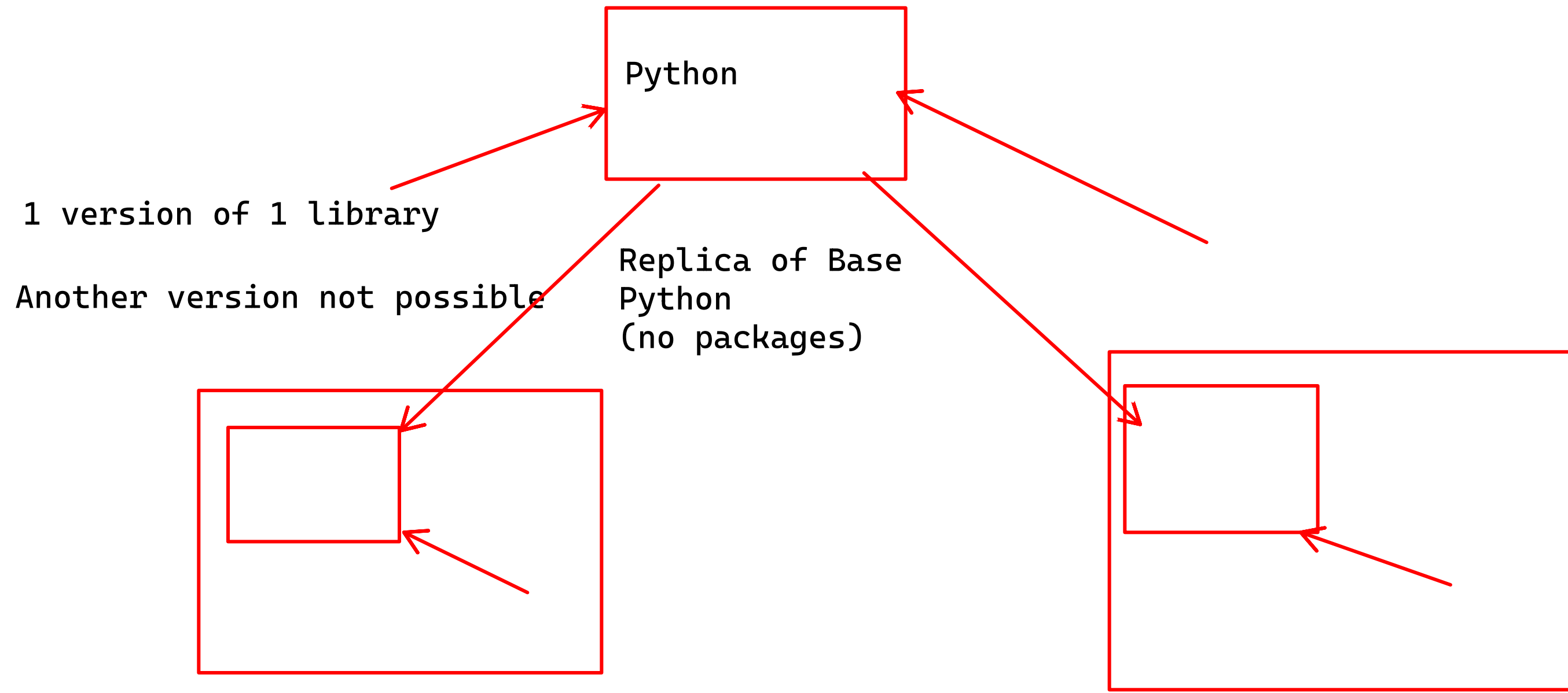
natural language

Copilot

Code Generation

files
code snippets
in the text editor

more context more helpful



```
python -m venv <folder_name>
```

```
python -m venv .venv
```

pip - python package manager
npm
maven/gradle
nuget
go get

pip

- installs softwares by default from pypi.org

Third Party Module

pydantic - data handling & Validation

numpy - numerical computations

pandas - datascience

matplotlib,

seaborn, - Visualization & Charting

altair charts

streamlit - dynamic dashboards

openpyxl - excels

docx2pdf

pptx2pdf

pptx2md

requests - connecting to internet & data extraction from apis

beautifulsoup - web scarping

Reportlab/fpdf - reports

py pandoc - python wrapper for pandoc tool

scipy -scientific computation

pil - image processing

opencv - videos processing

AIML Libraries

sklearn

keras

tensorflow

nltk

AI -Agents

langchain/langgraph

opencrew.ai

adk

Cloud

azure-cli sdk

boto3 -aws

SQLAlchemy - ORM Tool

Pandas

- data science library
- Series & dataframe
- Series : 1D Homogeneous
- DataFrame: 2D Heterogeneous

head() - top 5 rows

head(n) - top n rows

Data Pipeline



Explore more on

- Pandas
- matplotlib
- jinja2 templates
- reportlab

- langchain
- adk

Happy Learning & Happy Journey
All the Best !

Vishwas K Singh